



AI 赋能控制

Topic 例程 使用手册

V1.5.1

2026.1

文档版本

版本号	修订日期
1.4	2025 年 07 月 15 日
1.5	2025 年 07 月 23 日
1.5.1	2026 年 01 月 09 日

目录

文档版本II

1. 在 windows 中使用 topic..... 4

2. topic 程序说明..... 6

附录 7

1. 在 windows 中使用 topic

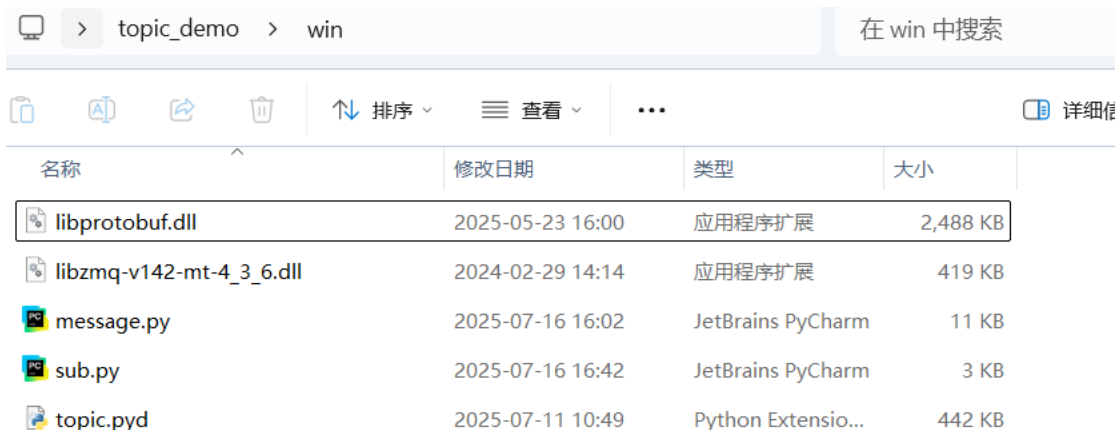
注：当前 python 版本只支持 python3.10 环境

(1) Windows 环境下在 \topic_demo\win 目录中有 topic.pyd 、 main.py、 message.py、 libprotobuf.dll、 libzmq-v142-mt-4_3_6.dll 五个文件

其中 main.py 是订阅端 python 的例程

message.py 是订阅端能打印的参数结构体

topic.pyd 是 topic 通信 python 的依赖库，另外两个 dll 文件则是 topic.pyd 的依赖库
依赖库可忽略，用户只需关注例程和参数结构体即可



The screenshot shows a Windows File Explorer window with the address bar set to 'topic_demo > win'. The search bar contains '在 win 中搜索'. The file list is as follows:

名称	修改日期	类型	大小
libprotobuf.dll	2025-05-23 16:00	应用程序扩展	2,488 KB
libzmq-v142-mt-4_3_6.dll	2024-02-29 14:14	应用程序扩展	419 KB
message.py	2025-07-16 16:02	JetBrains PyCharm	11 KB
sub.py	2025-07-16 16:42	JetBrains PyCharm	3 KB
topic.pyd	2025-07-11 10:49	Python Extensio...	442 KB

(2) sub_url 为发布端 ip 和端口，可根据需求更改

```
# 2. 配置节点选项
system_rtstate_options = topic.NodeOptions()
system_rtstate_options.node_name = 'test1'
system_rtstate_options.sub_url = 'tcp://localhost:19091'
print("Subscribing1 to:", system_rtstate_options.sub_url)

system_nrtstate_options = topic.NodeOptions()
system_nrtstate_options.node_name = 'test2'
system_nrtstate_options.sub_url = 'tcp://localhost:19091'
print("Subscribing2 to:", system_nrtstate_options.sub_url)
```

(3) 用户可根据自身需求在回调函数 display 函数后添加打印信息，参数见 message.py

```
# 1. 定义回调函数，处理接收到的 system_rtstate 消息
def on_system_rtstate_update(tt: topic.SystemRtState):
    print("\n" * 1)
    print("Received system rtstate update")
    parm_rt = message.SystemStateData()
    message.display_rt(tt, parm_rt)

    print(f"RT State - Header Timestamp: {parm_rt.header_timestamp}")
    print(f"RT State - Controller Name: {parm_rt.controller.controller_name}")

# 定义回调函数，处理接收到的 system_nrtstate 消息
def on_system_nrtstate_update(tt: topic.SystemNrtState):
    print("Received system nrtstate update")
    parm_nrt = message.SystemStateData()
    message.display_nrt(tt, parm_nrt)

    print(f"NRT State - System Running State: {parm_nrt.system_running_state}")
    print(f"NRT State - First Model Name: {parm_nrt.models[0].model_name}")
```

(4) message.py 中定义了可打印参数及其含义

```
sub.py  message.py x
1  from dataclasses import dataclass, field
2  from typing import List
3  import topic
4
5  @dataclass
6  class SlaveInfo:
7      slave_name: str = ""          # 从站名称
8      phy_id: int = 0               # 物理地址
9      alias: int = 0               # 逻辑别名
10     slave_state: int = 0          # 从站状态机
11     is_online: bool = False       # 是否在线
12     is_virtual: bool = False     # 是否为虚拟/仿真从站
13     is_error: bool = False       # 是否存在错误
```

(5) 运行效果如图，返回信息如图

```
z@z-virtual-machine:~/桌面/test/linux$ python3 sub.py
Subscribing1 to: tcp://localhost:19091
Subscribing2 to: tcp://localhost:19091
NodeOptions
  name: {test1}
  pub url: {}
  sub url: {tcp://localhost:19091}
  thread poll size: {1}
  pub queue length: {50}
  sub queue length: {1000}
init
on_monitor_started
zmq version 4.3.6
[SUB] connect: {tcp://localhost:19091}
sub_monitor thread started.
RTNode started successfully.
LoopRecv started...
NodeOptions
  name: {test2}
  pub url: {}
  sub url: {tcp://localhost:19091}
  thread poll size: {1}
  pub queue length: {50}
  sub queue length: {1000}
init
on_monitor_started
zmq version 4.3.6
[SUB] connect: {tcp://localhost:19091}
NRTNode started successfully.
Creating subscription for topic: system_rtstate
Creating subscription for topic: system_nrtstate
sub_monitor thread started.
LoopRecv started...
```

2. topic 程序说明

程序分为几个部分：配置、创建节点，订阅远程地址，订阅话题名称，回调函数打印话题信息。

创建 topic 节点，在回调函数中分别订阅"system_rtstate"及"system_nrtstate"主题来接收 消息。其中 system_rtstate 是实时接收，数据更新速度较快，system_nrtstate 是非实时接收，数据更新速度较慢。

程序中已经将所以接口信息全部打印，可以根据需求自行取用。

```

13 # 1. 定义回调函数，处理接收到的 system_rtstate 消息
14 > def on_system_rtstate_update(tt: topic.SystemRtState):...
92
93 # 定义回调函数，处理接收到的 system_nrtstate 消息
94 > def on_system_nrtstate_update(tt: topic.SystemNrtState):...
207
208 # system_rtstate中常用信息-快速使用
209 > def on_system_basicstate_update(tt: topic.SystemRtState):...
240
241 |
242 # 2. 配置节点选项
243 system_tpoic_options = topic.NodeOptions()
244 system_tpoic_options.node_name = 'test1'
245 system_tpoic_options.sub_url = 'tcp://192.168.2.153:19091'
246 safe_print("Subscribing1 to:", system_tpoic_options.sub_url)
247
248
249 # 3. 创建节点并启动
250 system_topic_node = topic.Node(system_tpoic_options)
251 if not system_topic_node.Start():
252 |     safe_print("Failed to start node.")
253 else:
254 |     safe_print("RTNode started successfully.")
255
256
257 # 4. 创建订阅
258 sub1 = system_topic_node.CreateSubscriptionRT("system_rtstate", on_system_rtstate_update)
259 sub2 = system_topic_node.CreateSubscriptionNRT("system_nrtstate", on_system_nrtstate_update)
260 sub3 = system_topic_node.CreateSubscriptionRT("system_rtstate", on_system_basicstate_update)
261
262
263 # 5. 保持程序运行，持续接收消息
264 try:
265 |     while True:
266 |         time.sleep(0.01) # 防止主线程退出

```

```

5 # 1. 定义回调函数，处理接收到的 system_rtstate 消息
6 def on_system_rtstate_update(tt: topic.SystemRtState):
7     print("\n" * 1)
8     print("Received system rtstate update")
9     print("Debug: on_system_rtstate_update called") # 调试日志
10    parm_rt = message.SystemStateData()
11    message.display_rt(tt, parm_rt)
12
13    print("***** System State Update *****")
14
15    print(f"""
16        1: header_timestamp: {parm_rt.header_timestamp}
17        2: header_frame_id: {parm_rt.header_frame_id}
18        3: system_running_state: {parm_rt.system_running_state}
19        4: controller_name: {parm_rt.controller.controller_name}
20        5: control_cycle: {parm_rt.controller.control_cycle}
21        6: global_count: {parm_rt.controller.global_count}
22        7: master_info: {parm_rt.controller.master_info}
23        8: is_link_up: {parm_rt.controller.is_link_up}
24    """)
25
26    b = 1
27    for model_info in parm_rt.models:
28        print(f"***** Models and Joints Information num: {b} *****")
29        print(f"model num:{b} 1: Model Name: {model_info.model_name}")
30        print(f"model num:{b} 2: Model Type: {model_info.model_type}")
31        b = b + 1
32

```

注：std::count 的打印方式没有线程保护，直接观察打印信息会存在顺序错乱问题属于正常现象，现有打印方式仅提供各接口以及调用方法，可根据需求进行选取调用。

附录

法律声明

1. 版权声明

© 2025 贞实科技版权所有。未经书面授权，禁止以任何形式复制、传播本文件内容。

2. 免责条款

使用者应自行承担因以下情况导致的损失风险：

- 未按指引操作造成的系统配置错误
- 擅自修改动态库文件引发的程序崩溃（如 message.dll 等库非授权替换）
- 开发环境版本不匹配导致的编译失败（如 CMake 2.8 与 VS2022 兼容问题）

3. 数据隐私条款（Data Privacy）

产品运行过程中产生的日志文件，仅包含设备状态信息，不会收集用户个人信息。